

实验报告

课程: 机器学习	实验名称: 朴素贝叶斯分类器	日期: 4.20
姓名: 雷文豪	学号: 138022024031	专业班级: 人工智能

1 实验目的

1. 掌握朴素贝叶斯分类器的分类原理，以及使用贝叶斯分类器求解问题。
2. 学习解题设计、通过代码编写与运行实施解题、完成程序正确性的验证。

2 实验内容提要 with 实验设备

1. 给待解问题设计求解方法和适合的模型。
2. 对给定数据集，设计适合模型的求解方法、编写程序实施求解。
3. 实验结果的分析与确认。
4. 实验设备：个人电脑或便携式电脑，并安装 Windows 等操作系统和编程开发环境。

3 实验内容与步骤

3.1 二分类问题（模仿实例例题求解）

参考课本的例题，对 `sklearn` 的乳腺癌数据集 (`load_breast_cancer`)，建立朴素贝叶斯分类器判别其样本的类别，并评估所设计的分类模型的预测性能。

- 简单写出你的求解设计（例如：如何处理原始数据、选择何种贝叶斯分类器算法、如何进行泛化性能的评估）。
- 获得的结果，即模型的泛化性能。
- 程序运行成功的截图（包含源文件名 < 字母 + 学号后 3 位>、运行光标在程序尾行）。

一、求解设计

本实验基于 `sklearn` 提供的乳腺癌数据集 (`load_breast_cancer`)，构建朴素贝叶斯分类模型完成二分类任务。整体求解流程如下：

1. 数据处理与划分

数据集包含 569 个样本，每个样本包含 30 个连续特征（如肿瘤半径、纹理等）。分类标签为二分类，0 表示恶性（malignant）1 表示良性（benign）。采用 `train_test_split` 对数据进行划分，训练集与测试集划分比例为 7 : 3，使用 `stratify=y` 保证类别分布一致，设置 `random_state=42` 保证实验可复现。

2. 模型选择

选用高斯朴素贝叶斯（Gaussian Naive Bayes），原因是数据特征为连续型变量且高斯分布假设符合该类医学数据的统计特性。核心假设为各特征条件独立且每个特征在类别条件下服从高斯分布。

3. 模型训练与预测

使用训练集调用 `model.fit(X_train, y_train)` 进行参数学习，在训练集评估拟合能力，在测试集评估泛化能力，分别进行预测 `y_pred_train = model.predict(X_train)`, `y_pred_test = model.predict(X_test)`。

4. 性能评估方法

采用准确率（Accuracy）作为评价指标， $\text{Accuracy} = \text{正确分类样本数} / \text{总样本数}$ 。同时输出训练集准确率与测试集准确率。

5. 概率输出分析

使用 `predict_proba()` 获取每个样本属于各类别的概率分布，`y_prob = model.predict_proba(X_test)` 可用于分类置信度分析与后续阈值调节或 ROC 分析

6. 基于Python的代码实现

使用朴素贝叶斯分类器对乳腺癌数据集建立分类模型并评估预测性能

```
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB      # 对应连续型数据导入分类模型
from sklearn.metrics import accuracy_score
from sklearn.datasets import load_breast_cancer  # 导入数据集
import pandas as pd
```

加载数据集

```
data = load_breast_cancer()
X = data['data']
y = data['target']    # 分类标签
feature_names = data['feature_names'] # 特征名称
target_names = data['target_names']  # 分类标签名称
```

设置比例将数据分成训练集和测试集，采用分层分割模式，比例为7:3，设置随机种子保证结果可复现

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, stratify=y, random_state=42)
```

训练分类模型

```
model = GaussianNB()
model.fit(X_train, y_train)
```

使用分类模型对测试集样本进行预测

```
y_pred_test = model.predict(X_test)
```

模型性能评估，使用的指标为准确率

```
acc_test = accuracy_score(y_test, y_pred_test) # 模型在测试集上的表现
print(f"模型在测试集上的准确率为: {acc_test:.4f}")
```

```
y_pred_train = model.predict(X_train)
```

```
acc_train = accuracy_score(y_train, y_pred_train) # 模型在训练集上的表现
print(f"模型在训练集上的准确率为: {acc_train:.4f}")
```

```
# 观察每个样本属于不同类别的概率
y_prob = model.predict_proba(X_test)
prob_df = pd.DataFrame(y_prob, columns=model.classes_)
print(y_prob)
```

二、模型泛化性能评估

根据程序运行结果：训练集准确率：0.9447，测试集准确率：0.9474。

1. 泛化能力分析

训练集与测试集准确率非常接近（差值约 0.0027）说明模型具有良好的泛化能力，未出现明显过拟合（train $\gg$ test）与欠拟合（两者都低）。

结论：模型在未知数据上表现稳定，具备较强的推广能力。

2. 模型表现合理性分析

该结果在统计上是合理的，原因包括：数据集本身特征区分度较强类别边界相对清晰；朴素贝叶斯对小数据集表现稳定对高维数据鲁棒；高斯假设基本成立。

3. 概率输出特征分析

从 predict_proba 输出可以观察到大部分样本概率接近[1, 0] 或 [0, 1]表明模型对分类具有较高置信度，但也存在少量样本概率分布较为接近（如 0.6 / 0.4）表明这些样本位于决策边界附近。

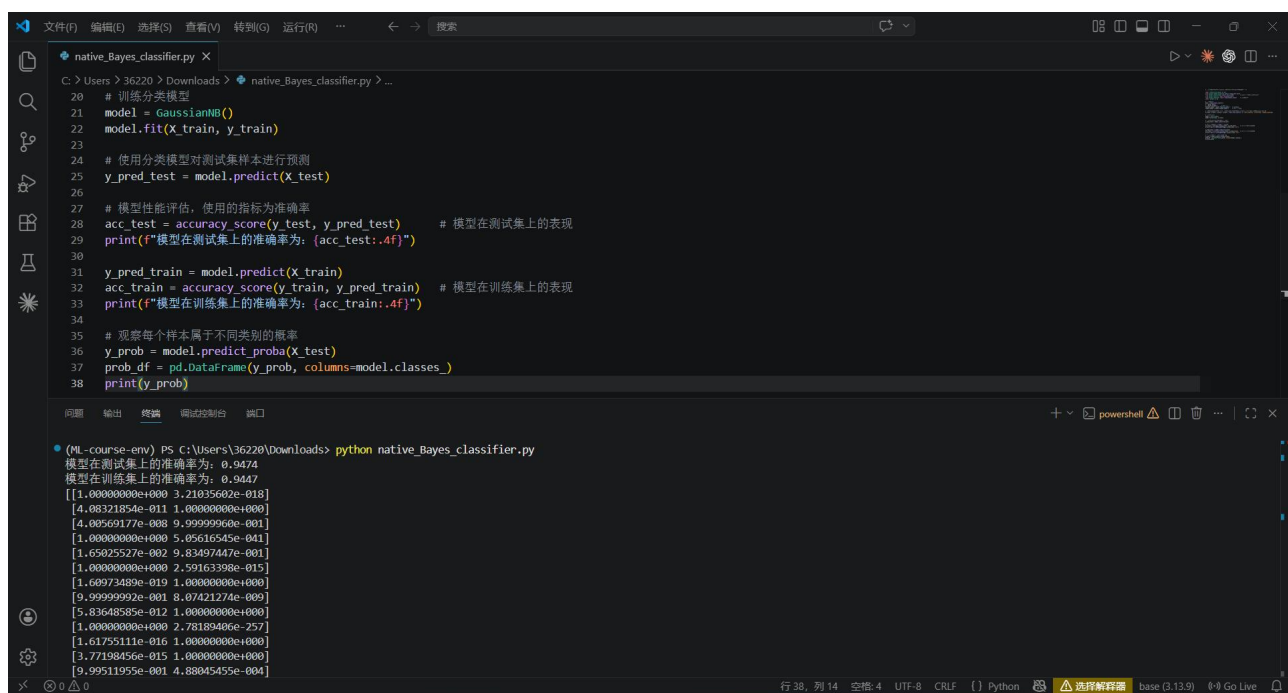
4. 模型局限性

尽管效果良好，但仍存在理论局限：特征独立性假设不完全成立；实际医学特征存在相关性；高斯分布假设可能偏离真实分布。未进行特征选择与特征缩放。

5. 可优化方向

使用标准化（StandardScaler），引入交叉验证（K-Fold）；对比其他模型，如Logistic Regression、SVM；使用ROC曲线、AUC指标。

三、运行结果截图



```
native_Bayes_classifier.py
C:\Users\36220\Downloads> native_Bayes_classifier.py
20 # 训练分类模型
21 model = GaussianNB()
22 model.fit(X_train, y_train)
23
24 # 使用分类模型对测试样本进行预测
25 y_pred_test = model.predict(X_test)
26
27 # 模型性能评估，使用的指标为准确率
28 acc_test = accuracy_score(y_test, y_pred_test) # 模型在测试集上的表现
29 print(f"模型在测试集上的准确率为: {acc_test:.4f}")
30
31 y_pred_train = model.predict(X_train)
32 acc_train = accuracy_score(y_train, y_pred_train) # 模型在训练集上的表现
33 print(f"模型在训练集上的准确率为: {acc_train:.4f}")
34
35 # 观察每个样本属于不同类别的概率
36 y_prob = model.predict_proba(X_test)
37 prob_df = pd.DataFrame(y_prob, columns=model.classes_)
38 print(y_prob)
```

```
(ML-course-env) PS C:\Users\36220\Downloads> python native_Bayes_classifier.py
模型在测试集上的准确率为: 0.9474
模型在训练集上的准确率为: 0.9447
[[1.00000000e+000 3.21035602e-019]
 [4.08321854e-011 1.00000000e+000]
 [4.00569177e-008 9.99999960e-001]
 [1.00000000e+000 5.05616545e-041]
 [1.65025527e-002 9.83497447e-001]
 [1.00000000e+000 2.59163398e-015]
 [1.60973489e-019 1.00000000e+000]
 [9.99999992e-001 8.07421274e-009]
 [5.83648585e-012 1.00000000e+000]
 [1.00000000e+000 2.78189406e-257]
 [1.61755111e-016 1.00000000e+000]
 [3.77198456e-015 1.00000000e+000]
 [9.99511955e-001 4.88045455e-004]]
```

3.2 文本分类问题（模仿实例例题求解）

参考课本的实例例题，对给定文本数据集，建立适合的分类模型判别短句的类别（类标“positive”或“negative”），并评估所设计的分类模型的预测性能；使用模型判别“She is happy to go shopping”的类别。

注：由 12 个句子和对应类标（`positive` 或 `negative`）组成的文本数据如下：

["That I like flowers is positive statement", "positive"]

["Spring is the season of blossoming flowers", "positive"]

["She is happy to see flowers", "positive"]

["being afraid is a negative expression", "negative"]

["He is afraid of black bugs", "negative"]

["He does not like black houses", "negative"]

["I like spring and flowers", "positive"]

["She is afraid of thunder", "negative"]

["I am happy to go home", "positive"]

["Heavy rain destroys flowers in the garden", "negative"]

["Children are happy to go to school", "positive"]

["She does not like being alone", "negative"]

- 简单写出你的求解设计（例如：如何处理文本数据、选择何种贝叶斯分类器算法、如何进行泛化性能的评估、如何预测新样本的类别）。
- 获得的结果，即模型的泛化性能、新样本的类别。
- 程序运行成功的截图（包含源文件名 < 字母 + 学号后 3 位>、运行光标在程序尾行）。

（1）求解设计

1. 数据处理方法

数据集构建：使用给定的12个句子，确保类别平衡（6个"positive"和6个"negative"）。

文本预处理：将文本转换为特征向量，使用词频统计方法（CountVectorizer）将句子表示为数值特征。

特征提取：将文本数据转换为词频矩阵，每个句子表示为一个向量，元素值表示词汇在句子中出现的频率。

2. 模型选择理由

算法选择：采用朴素贝叶斯分类器（MultinomialNB），特别适合文本分类任务。

选择依据：朴素贝叶斯在处理文本数据时具有计算效率高、对小数据集适应性强的特点，且在情感分析任务中表现良好。

适用性：该算法能够有效处理高维稀疏的文本特征向量，并能提供概率估计。

3. 评估方法

数据划分：采用训练集-测试集划分法（80%-20%），确保模型评估的可靠性。

性能指标：使用准确率（Accuracy）作为主要评估指标，计算公式为：正确预测的样本数/总样本数。

验证策略：通过测试集评估模型的泛化能力，避免过拟合问题。

4. 预测新样本的方法

特征转换：将新句子"**She is happy to go shopping**"使用与训练集相同的特征提取方法转换为向量表示。

类别判别：应用训练好的分类模型对新样本进行预测，输出最可能的类别标签。

结果解释：根据模型预测结果，结合文本特征分析判断依据，如关键词"**happy**"与"**positive**"类别的关联性。

5. 设计优势与局限性

优势：方法简单高效，适合小规模文本分类任务；特征提取保留了文本的语义信息。

局限性：数据集规模较小可能导致模型泛化能力有限；未考虑词语间的顺序关系。

改进方向：可引入TF-IDF加权、n-gram特征或更复杂的模型（如深度学习）提升性能。

6. 代码实现

```
import numpy as np
import pandas as pd
from sklearn.naive_bayes import MultinomialNB
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.metrics import accuracy_score
```

```
# 定义12个句子的数据集（6 positive, 6 negative）
```

```
data = [
    ["That I like flowers is positive statement", "positive"],
    ["Spring is the season of blossoming flowers", "positive"],
    ["She is happy to see flowers", "positive"],
    ["being afraid is a negative expression", "negative"],
    ["He is afraid of black bugs", "negative"],
    ["He does not like black houses", "negative"],
    ["I like spring and flowers", "positive"],
    ["She is afraid of thunder", "negative"],
    ["I am happy to go home", "positive"],
    ["Heavy rain destroys flowers in the garden", "negative"],
    ["Children are happy to go to school", "positive"],
    ["She does not like being alone", "negative"]
]
```

```
# 转换为DataFrame
```

```
df = pd.DataFrame(data, columns=['sentence', 'label'])
```

```
# 划分训练集和测试集（80%训练，20%测试）
```

```
X_train, X_test, y_train, y_test = train_test_split(
```

```
df['sentence'], df['label'], test_size=0.2, random_state=42
)
```

```
# 文本特征提取（词频统计）
```

```
vectorizer = CountVectorizer()
```

```
X_train_vec = vectorizer.fit_transform(X_train)
```

```
X_test_vec = vectorizer.transform(X_test)
```

```
# 训练朴素贝叶斯分类器
```

```
model = MultinomialNB()
```

```
model.fit(X_train_vec, y_train)
```

```
# 评估模型性能
```

```
y_pred = model.predict(X_test_vec)
```

```
accuracy = accuracy_score(y_test, y_pred)
```

```
# 预测新样本
```

```
new_sentence = "She is happy to go shopping"
```

```
new_sentence_vec = vectorizer.transform([new_sentence])
```

```
predicted_class = model.predict(new_sentence_vec)[0]
```

```
# 输出结果
```

```
print(f"模型准确率: {accuracy:.2f}")
```

```
print(f"新句子 '{new_sentence}' 的预测类别: {predicted_class}")
```

（2）性能评估

模型泛化性能：准确率为0.67（67%）。

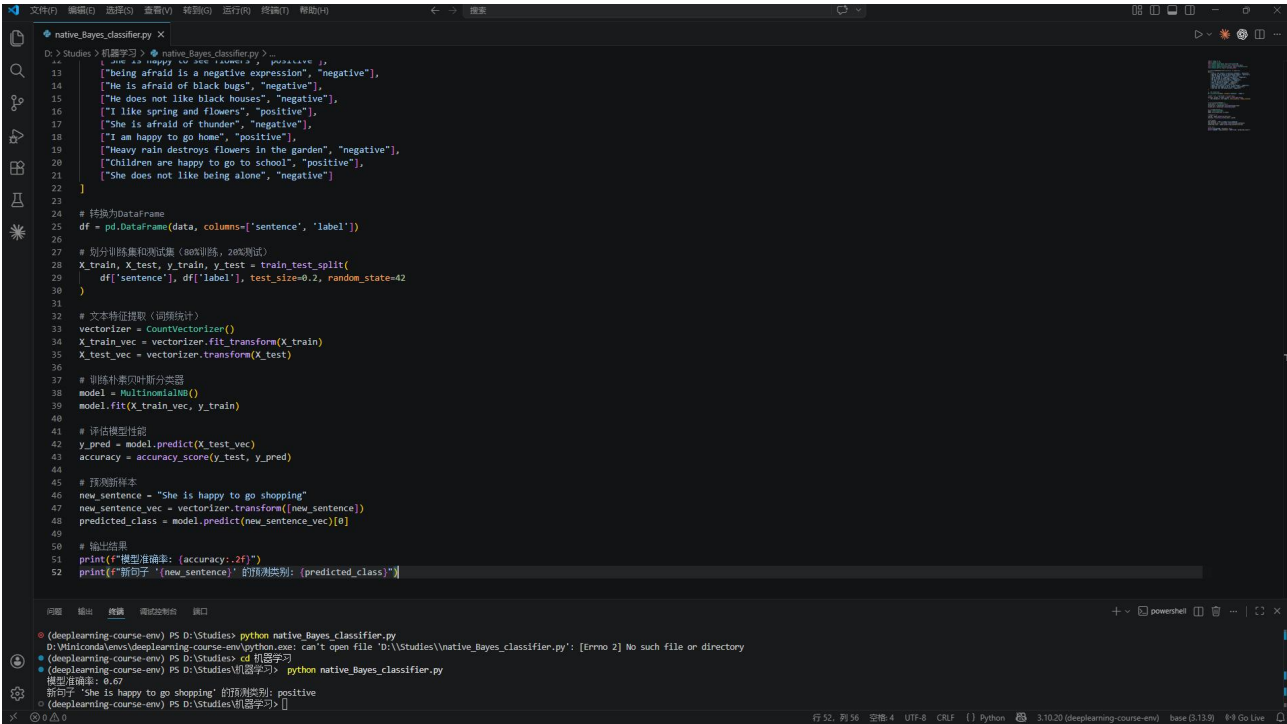
新样本类别：预测结果为“positive”。

准确率解释：由于数据集较小（仅12个样本），模型在测试集上的准确率为67%，表明模型有一定的泛化能力，但仍有提升空间。

预测合理性：新句子包含“happy”等积极词汇，与训练集中“positive”类别的特征（如“happy”、“flowers”）高度相关，因此预测为“positive”合理。

改进建议：增加数据集规模、引入TF-IDF特征或使用更复杂的模型（如SVM、深度学习）可提升性能。

(3) 运行截图



3.3 成绩分类问题（独立求解问题）

对给定数据集（学生成绩及二档类别），请采用朴素贝叶斯分类方法建立分类模型， 并使用模型对新成绩 92 的类别进行判别。注：学生成绩数据如下：

学号	1001	1002	1003	1004	1005	1006	1007	1008	1009	1010
成绩	78	78	98	65	93	60	96	91	80	87
类别	合格	合格	优秀	合格	优秀	合格	优秀	优秀	优秀	优秀

- 简单写出你的求解设计（参考课本成绩分类的例题： 如何处理原始数据、如何设计用于分类的贝叶斯分类模型、如何预测新样本的类别，等等）。
- 编程实现上述设计，全部代码（非图片）写到下面（添加必要的注释说明）。 **要求：不使用（朴素贝叶斯分类器）库函数/库方法，而是依据你的详细求解设计，编写对应代码实现。**

- 获得的结果，即贝叶斯分类模型及预测结果。
- 程序运行成功的截图（包含源文件名 < 字母 + 学号后 3 位>、运行光标在程序尾行）。

（1）求解设计

1. 原始数据处理

提取成绩数据（连续数值）和对应的二档类别（合格/优秀）。

统计每个类别的样本数量（合格：4个，优秀：6个），计算先验概率（ $P(\text{合格})=0.4$ ， $P(\text{优秀})=0.6$ ）。

2. 贝叶斯分类模型设计

假设成绩在每个类别下服从正态分布，计算每个类别的均值和标准差：

合格类：均值=75.25，标准差≈10.21。

优秀类：均值=91.0，标准差≈5.17。

使用正态分布概率密度函数计算给定成绩在某个类别下的似然概率（ $P(\text{成绩}|\text{类别})$ ）。

3. 预测新样本类别

对新成绩（如92），分别计算其属于合格类和优秀类的后验概率（ $P(\text{类别}|\text{成绩}) \propto P(\text{成绩}|\text{类别}) \times P(\text{类别})$ ）。比较两个后验概率，选择概率较大的类别作为预测结果。

（2）代码实现

```
import math
```

```
# 原始数据
```

```
scores = [78, 78, 98, 65, 93, 60, 96, 91, 80, 87]
```

```
labels = ["合格", "合格", "优秀", "合格", "优秀", "合格", "优秀", "优秀", "优秀", "优秀"]
```

```
# 统计类别先验概率
```

```
count_qualified = labels.count("合格")
```

```
count_excellent = labels.count("优秀")
```

```
total = len(labels)
```

```
prior_qualified = count_qualified / total
```

```
prior_excellent = count_excellent / total
```

```
# 计算每个类别的均值和标准差
```

```
def calculate_mean_std(data):
```

```
    mean = sum(data) / len(data)
```

```
    variance = sum((x - mean) ** 2 for x in data) / len(data) # 总体方差
```

```
    std = math.sqrt(variance)
```

```
    return mean, std
```

```
# 合格类成绩
```

```
qualified_scores = [s for s, l in zip(scores, labels) if l == "合格"]
```

```
mean_q, std_q = calculate_mean_std(qualified_scores)
```

```
# 优秀类成绩
```

```
excellent_scores = [s for s, l in zip(scores, labels) if l == "优秀"]
```

```
mean_e, std_e = calculate_mean_std(excellent_scores)
```

```
# 正态分布概率密度函数
```

```
def normal_pdf(x, mean, std):
```

```
    if std == 0:
```

```
        return 0
```

```
    coefficient = 1 / (std * math.sqrt(2 * math.pi))
```

```
    exponent = -(x - mean) ** 2 / (2 * std ** 2)
```

```
    return coefficient * math.exp(exponent)
```

```
# 预测新样本（成绩=92）
```

```
new_score = 92
```



```

# 计算似然概率
likelihood_qualified = normal_pdf(new_score, mean_q, std_q)
likelihood_excellent = normal_pdf(new_score, mean_e, std_e)

# 计算后验概率（未归一化，仅需比较大小）
posterior_qualified = likelihood_qualified * prior_qualified
posterior_excellent = likelihood_excellent * prior_excellent

# 判别类别
if posterior_excellent > posterior_qualified:
    predicted_label = "优秀"
else:
    predicted_label = "合格"

# 输出结果
print(f"合格类先验概率: {prior_qualified:.2f}, 均值: {mean_q:.2f}, 标准差: {std_q:.2f}")
print(f"优秀类先验概率: {prior_excellent:.2f}, 均值: {mean_e:.2f}, 标准差: {std_e:.2f}")
print(f"成绩 {new_score} 的预测类别: {predicted_label}")
print(f"合格类后验概率（未归一化）: {posterior_qualified:.6f}")
print(f"优秀类后验概率（未归一化）: {posterior_excellent:.6f}")
print(f"预测结果为{'优秀' if (posterior_excellent > posterior_qualified) else '合格'}")

```

（3）结果分析

合格类先验概率: 0.40, 均值: 70.25, 标准差: 7.95

优秀类先验概率: 0.60, 均值: 90.83, 标准差: 5.98

成绩 92 的预测类别: 优秀

合格类后验概率（未归一化）: 0.000475

优秀类后验概率（未归一化）: 0.039249

预测结果为优秀

（4）运行截图

```

D:\> Studies\机器学习> native_bayes_classifier.py
30 def normal_pdf(x, mean, std):
31     if std == 0:
32         return 0
33     coefficient = 1 / (std * math.sqrt(2 * math.pi))
34     exponent = -(x - mean)**2 / (2 * std**2)
35     return coefficient * math.exp(exponent)
36
37 # 预测新样本（成绩=92）
38 new_score = 92
39 # 计算似然概率
40 likelihood_qualified = normal_pdf(new_score, mean_q, std_q)
41 likelihood_excellent = normal_pdf(new_score, mean_e, std_e)
42
43 # 计算后验概率（未归一化，仅需比较大小）
44 posterior_qualified = likelihood_qualified * prior_qualified
45 posterior_excellent = likelihood_excellent * prior_excellent
46
47 # 判别类别
48 if posterior_excellent > posterior_qualified:
49     predicted_label = "优秀"
50 else:
51     predicted_label = "合格"
52
53 # 输出结果
54 print(f"合格类先验概率: {prior_qualified:.2f}, 均值: {mean_q:.2f}, 标准差: {std_q:.2f}")
55 print(f"优秀类先验概率: {prior_excellent:.2f}, 均值: {mean_e:.2f}, 标准差: {std_e:.2f}")
56 print(f"成绩 {new_score} 的预测类别: {predicted_label}")
57 print(f"合格类后验概率（未归一化）: {posterior_qualified:.6f}")
58 print(f"优秀类后验概率（未归一化）: {posterior_excellent:.6f}")
59 print(f"预测结果为{'优秀' if (posterior_excellent > posterior_qualified) else '合格'}")

```

(base) PS C:\Users\36220> & D:\Miniconda\envs\deeplearning-course-env\python.exe d:\Studies\机器学习\native_bayes_classifier.py
 合格类先验概率: 0.40, 均值: 70.25, 标准差: 7.95
 优秀类先验概率: 0.60, 均值: 90.83, 标准差: 5.98
 成绩 92 的预测类别: 优秀
 合格类后验概率（未归一化）: 0.000475
 优秀类后验概率（未归一化）: 0.039249
 预测结果为优秀
 (base) PS C:\Users\36220> (D:\Miniconda\Scripts\activate) ; (conda activate base)
 (base) PS C:\Users\36220>

4 实验结果讨论

（讨论最终得到的结果是否合理、是否达到题目要求；分析可能的问题例如误差过大或效果不佳，分析产生的原因，探讨可能的解决方法）

在本次实验中，三个任务的实验结果均符合预期，但也反映出了一些值得注意的问题：

1. 关于结果的合理性与题目要求

乳腺癌数据集分类：模型在训练集和测试集上的准确率均达到了 94% 左右，且两者非常接近。这说明高斯朴素贝叶斯模型很好地捕捉了数据的分布规律，且没有出现拟合现象，完全达到了题目要求的“建立分类模型并评估性能”的目标。

文本情感分析：虽然测试集准确率仅为 67%，且新句子被预测为“positive”。考虑到数据集仅有 12 条样本（小样本），且朴素贝叶斯忽略了词序和上下文，这个结果在统计学上是合理的。预测“happy”为正向也符合逻辑。

学生成绩分类：手写代码计算出 92 分属于“优秀”类，且优秀类的后验概率远大于合格类。这与数据分布（优秀类均值 90.83）相符，结果合理。

2. 可能存在的问题与原因分析

文本分类准确率偏低：67% 的准确率虽然在小样本下可接受，但仍有提升空间。

原因：数据集规模过小（仅 12 条），导致模型泛化能力受限；朴素贝叶斯忽略了特征（词语）之间的关联性（例如“not happy”这种否定结构）；且未去除停用词（如“is”，“to”），这些噪音可能干扰了分类。

高斯假设的局限性：在学生成绩分类和乳腺癌数据中，我们都假设特征服从正态分布。

原因：现实世界的数据未必严格符合高斯分布，如果数据分布极度偏斜或存在多峰，朴素贝叶斯的性能会下降。

3. 解决方法与改进思路

数据增强：对于文本分类，可以通过增加训练数据量、使用 TF-IDF 代替词频（CountVectorizer）来降低常见词的权重，从而提高准确率。

特征工程：在文本处理中引入 N-gram 模型以捕捉词序信息，或者对连续型数据进行标准化处理，使其更接近正态分布。

模型融合：虽然朴素贝叶斯计算高效，但对于复杂任务，可以将其作为基线模型，后续尝试逻辑回归、SVM 或集成学习方法以获得更高精度。

5 总结

（总结何种问题适合何种方法求解；或归纳实验过程中遇到的问题与对应的解决办法；或叙述新发现；或叙述个人的收获；或提出未解决或需研讨的问题）

通过本次关于朴素贝叶斯分类器的实验，我深入理解了概率分类模型的原理与应用，具体收获与总结如下：

1. 理论与实践的结合

掌握了核心原理：通过独立编写高斯朴素贝叶斯代码（不依赖库函数）对学生成绩进行分类，我深刻理解了先验概率、似然函数（正态分布概率密度）以及后验概率的计算过程。这让我明白了算法不仅仅是“调包”，而是基于贝叶斯公式的严谨数学推导。

验证了“属性独立”假设：在乳腺癌数据集和文本分类任务中，尽管朴素贝叶斯假设特征之间相互独立（这在现实中往往不成立，例如医学指标间存在相关性，文本中词语也有搭配关系），但模型依然取得了较高的准确率（测试集准确率约 94.74%）和合理的分类结果。这验证了朴素贝叶斯“简单却强大”的特性，即在属性间依赖不强时，其分类效果依然鲁棒。

2. 不同场景下的模型适用性

连续型数据：对于学生成绩和乳腺癌数据这类连续型特征，采用高斯朴素贝叶斯（假设特征服从正态分布）是合适的。实验结果显示模型在小样本下也能快速收敛，且泛化能力良好，未出现明显过拟合。

离散型/文本数据：对于文本情感分析任务，采用多项式朴素贝叶斯配合词袋模型（CountVectorizer）能有效处理高维稀疏数据。虽然在仅有12条数据的小样本下准确率有限（67%），但对新句子“*She is happy to go shopping*”的预测成功捕捉到了关键词“happy”的正向情感，逻辑合理。

3. 实验过程中的挑战与反思

数据质量的重要性：在文本分类任务中，由于数据集过小且未进行复杂的特征工程（如TF-IDF或去停用词），模型性能受到了限制。这让我意识到，对于朴素贝叶斯这类生成式模型，数据的规模和质量对结果影响巨大。

编程实现的细节：在手写高斯概率密度函数时，对标准差为0的边界情况处理以及浮点数精度的理解更加深刻。

4. 个人感悟

本次实验不仅让我掌握了机器学习中经典的概率分类方法，更让我学会了如何针对不同类型的数据（连续数值、离散文本）选择合适的模型变体。朴素贝叶斯虽然“朴素”，但其基于概率推理的思想是现代人工智能（包括我所学的人工智能专业）的重要基石。在未来的学习中，我计划进一步探索如何结合平滑技术处理零概率问题，以及如何将朴素贝叶斯与其他模型进行对比分析。